Abstraction Project Notes

Brandon Tang

Symbolic Simulation for Functional Properties

The input to symbolic trajectory evaluation consists of an antecedent and a list of output constraints.

- The antecedent maps BDD variables to input wires in the circuit
- The output constraints map BDD expressions to signals in the circuit
  - These allow us to assert if a certain wires should be high/low/either at a certain time step for given conditions based on the antecedent

These were traditionally written in the form of LTL formulae, but have been rewritten into a 4 tuple form in VOSSII as follows:

$$[(\text{signal}, \text{tickStart} : \text{tickEnd}, \text{highExpr}, \text{lowExpr})]$$

This is equivalent to the LTL formula `N^tick ((highExpr -> (signal is 1) and lowExpr -> (signal is 0))) for each tick in the range`.

For an antecedent, this means that we will set the signal to be true if the `highExpr` is true and set the signal to be false if the `lowExpr` is true. Otherwise the signal is `X`.

For output constraints this means that when the assignment of BDD variables make `highExpr` true, we need the signal to be high from `tickStart` to `tickEnd`. Similarly for `lowExpr`. Thus each output constraint tuple can be split into 2 parts: the positive constraint where we need the signal to be true under a certain condition, and the negative constraint where we need the signal to be false under a certain condition. We let the `highExpr` and `lowExpr` be termed as the guards of the constraint.

Jaspergold fully implements symbolic simulation without support for abstraction with an indexing relation. Here, antecedents are not dual rail but rather just singular expressions, this means that we have no symbolic indexing and abstraction beyond the inital Xs present in the circuit. The output constraints here also do not have guards. To conditionally check properties, we would adjust the input constraint `cin`.

For the purposes of doing symbolic indexing, we settled on using the 4-tuple form of antecedents and output. This makes it easier to port over the existing theory on indexing transformations from the STE papers. Furthermore, Jasper's symbolic simulation engine accepts stimuli in the form of the 4-tuple antecedent.

Indexing Relations

Indexing relations are used to represent abstractions of circuit inputs. This means that they merge sets of circuit inputs into cases.

Indexing relations $R[X, C, T]$ are boolean formulae over BDD variables where

- X are the indexing BDD variables that perform the symbolic indexing
- T are the target BDD variables that are indexed. These correspond to input wires in the circuit
- C are the symbolic constants that correspond to target wires that should not be indexed

The indexing relation can be interpreted as follows:

- For each X, C, we cover the cases where $\exists T R[X, C, T]$.
- Since the indexing relation can be a many to many relation, we can have multiple T, C cases covered by a single X, C case indexing.

Preimage Operations

We define some operations involving the indexing relation here:

Suppose
- $P[C, T]$ represents some set of input cases (e.g. (C, T) is in the set iff P[C, T] is true)
- $R[X, C, T]$ is an indexing relation

The **weak preimage** of P under R,

$$P_R[X, C] = \exists T(R[X, C, T] \wedge P[C, T])$$

is the set of (X, C) that cover at least one (T, C) case in P

The **strong preimage** of P under R,

$$P^R[X, C] = P_R \wedge \neg\exists T(R[X, C, T] \wedge \neg P[C, T])$$

is the set of (X, C) that cover at least one case in P and don't cover any cases not in P.

Image Operation
Taking the image of $H[X, C]$ returns the cases that are indexed by $H[X, C]$ under R.

$$\text{im}(H, R)[C, T] = \exists X(R[X, C, T] \wedge H[X, C])$$

While this is not directly used for finding proofs, it is useful for extracting counter examples for disproven properties.

Relationship between Preimages
For any fixed R and any given P and (X, C) we have one of the following cases:
- X, C indexes cases only in P
- X, C indexes cases only in $\neg P$
- X, C indexes cases in both P and $\neg P$
- X, C indexes cases in neither P nor $\neg P$

Any X, C that fall into the 4th category must correspond to empty indexing cases, i.e. don't map onto any (C, T) at all. Notice that this is independent of P, thus, we term the union of the first 3 cases as the domain of R.

$$\text{dom}(R)[X, C] = \exists T(R[X, C, T])$$

Observe that
- The weak preimage is the set of X, C that fall into the 1st and 3rd categories.
- The strong preimage is the set of X, C that fall into the 1st category.

We can note that the following are then also equivalent definitions of the strong preimage operation:

$$\begin{aligned} P^R[X, C] &= \text{dom}(R) \wedge \overline{\overline{P}^R} \\ &= \text{dom}(R) \wedge (\forall T(R[X, C, T] \to P[C, T])) \end{aligned}$$

Indexing Coverage Condition
For each property that we wish to check, we let the high and low expressions be P_i and Q_i respectively. The total space of inputs we need to check is then $B[C, T] = \bigvee_i (P_i \vee Q_i)$. In order for our abstraction to properly cover the space of required inputs, we require that

$$\forall T \forall C(B[T, C] \to \exists X(R[X, C, T]))$$

Generally we will be having guards that are equivalent to true, i.e. should hold for all inputs. In this case, $B[C, T] \equiv \text{True}$ so we coverage condition takes the simpler form:

$$\forall T \forall C \exists X (R[X, C, T])$$

Indexing Transformation and Symbolic Simulation Procedure

Suppose we are given an indexing relation, `idx_rel`, an antecedent list and an output constraint list. We will describe the procedure to prove that the output constraints hold.

We first apply the strong preimge operations on the high and low expressions of the antecedent tuples as follows:

```
antv = [(signal, tickStart:tickEnd, strongPreimage idx_rel highExpr, strongPreimage idx_rel lowExpr)]
```

Next we run the symbolic simulator with the transformed antecedent. This produces an evaluation sequence which will contain `sigHighExpr` and `sigLowExpr` BDD expressions for when a signal at a certain time step will definitely be high or low. If for a given assignment, neither of these expressions are true, then the signal is unknown. For a given signal and time step s, we let the high expression be H_s and the low expression be L_s.[1]

From this evaluation sequence, we can then check the output constraints. Suppose want to check a output constraint of the form `(signal, tickStart:tickEnd, P, Q)` where $P[C, T]$ and $Q[C, T]$ are the BDD expressions that correspond to inputs on which a signal should be high and low respectively. We will transform the output constraint to `(signal, tickStart:tickEnd, P_R, Q_R)` where P_R and Q_R are the weak preimages of P and Q under the indexing relation.

Then we analyse the `highexpr` and `lowexpr` BDD expressions from the evaluation sequence for the signal at each of the ticks in the tick range. Suppose that at a given tick t, for the property wire we have some the high expression as $H[X, C]$ and the low expression as $L[X, C]$. We term the high expression of the signal as the "residual".

We will check if $(P_R \rightarrow H)[X, C] \equiv \text{True}$ and whether $(Q_R \rightarrow L)[X, C] \equiv \text{True}$. If both of these hold, then the output constraint is satisfied.

If we have $(P^R \wedge L) \neq \text{False}$ it means that some indexing cases that only index into P actually cause s to be low, this is a counter example to the positive part of property so the positive part of property is disproven. We can inspect the cases indexed as $\text{im}(P^R \wedge L, R)$ to get the actual counter example in terms of the original circuit inputs.

Similarly, if we have $(Q^R \wedge H) \neq \text{False}$, we have disproven the negative part of the property.

It is possible to not prove the property but also have no counter examples. This can happen in cases where the antecedent does not provide enough information to prove or disprove the property. This is called a weak disagreement and requires changing the indexing relation to obtain a proof/counter example.

Correctness of Indexing Transformation and Output Constraint Checking

We prove that the above procedure for indexing transformation and output constraint checking is correct.

Here we only need to focus on proving the correctness of positive output constraint checking, i.e. that under a certain condition, a signal will be high. The proof for the negative output constraint checking is analagous.

[1] We will sometimes drop the subscript when there is only 1 relevant signal and time step being considered.

With our formulation of relational properties later on, we don't even need to deal with guards, but our proof of correctness will deal with the general case since guards can help with environmental constraints (discussed later).

Symbolic Simulation Invariant

Consider some residual $H_s[X, C]$ in a signal $s(C, T)$ at a fixed time step. We always have the fact that

$$H_s[X, C] \wedge R[X, C, T] \rightarrow s(C, T)$$

(For all X, C, T)

This can be proved via induction on the fan-in of S.

The basecase is where s is an input or state variable corresponding to a certain time step, with `highexpr` P. In this case, since we apply the strong preimage to the antecedents, we have that

$$H = P^R = \text{dom}(R) \wedge \forall T(R[X, C, T] \rightarrow P[C, T])$$

This directly gives us that for all X, C, T where $H[X, C] \wedge R[X, C, T]$, we have $P[C, T]$. But $s(C, T) = P[C, T]$ so we are done.[2]

An inductive case example:

Suppose $s = s_1 \wedge s_2$. We have Q_1 and Q_2 as the residuals of s_1 and s_2 respectively.

- $Q_1[X, C] \wedge R[X, C, T] \rightarrow s_1(C, T)$
- $Q_2[X, C] \wedge R[X, C, T] \rightarrow s_2(C, T)$
- So $Q_1[X, C] \wedge Q_2[X, C] \wedge R[X, C, T] \rightarrow s_1 \wedge s_2$
- But is equivalent to $H[X, C] \wedge R[X, C, T] \rightarrow s$ since $Q = Q_1 \wedge Q_2$ via the symbolic simulator

Output Constraint Checking

Output constraints that we wish to prove are of the form:

$$\forall C \forall T(P[T, C] \rightarrow s(C, T))$$

If we have that $P_R[X, C] \rightarrow H[X, C]$ and the coverage condition $\forall T \forall C(B[C, T] \rightarrow \exists X(R[X, C, T]))$ holds, then this will be true.

First remember that $P_R = \exists T(R[X, C, T] \wedge P[C, T])$.

Fix some C, T such that $P[C, T]$ is true.

- Since $P[C, T]$ is true, then $\forall X(R[X, C, T] \rightarrow P_R[X, C])$ by the definition of P_R.
- Since $P[C, T]$ is true, so is $B[C, T]$
- Consider some X^* such that $R[X^*, C, T]$ is true, this must exist by the coverage condition
- Since $P[T, C]$ and $R[X^*, C, T]$ are true, then we must have $P_R[X^*, C]$
- But now since $P_R[X, C] \rightarrow H[X, C]$, we have $H[X^*, C]$ as well
- Using the symbolic simulation invariant, since $H[X^*, C]$ and $R[X^*, C, T]$ are true, then $s(C, T)$ is true
- Since this did not rely on the values of C, T, then we have that $\forall C \forall T(P[T, C] \rightarrow s(C, T))$ is true

Alternative Checking by Reversing the Indexing

Another way to do the check is to take the image of the residual under the indexing relation, i.e. $\text{im}(H, R)[C, T]$. This will symbolically represent all the cases for which we know the property will hold.

[2]Note that we don't use the domain part for this. Indeed, the analysis will still work, but this serves to remove useless/inconsistent indexing cases. This is useful for counterexample analysis later on.

$$
\begin{aligned}
C, T \in \mathrm{im}(H, R) &\Rightarrow \exists X(H[X, C] \wedge R[X, C, T]) \\
&\Rightarrow H[X^*, C] \wedge R[X^*, C, T] \text{ for some } X^* \\
&\Rightarrow s(C, T) \text{ by symbolic simulation invariant}
\end{aligned}
$$

We can then check that $P \rightarrow \mathrm{im}(H, R) \equiv \mathrm{True}$. If so then the property is true. On some circuits and indexing relations, this method can avoid weak disagreements compared to the above method. This is particularly true if some cases in P are indexed by multiple X, C. However, if the number of indexing variables is much less than the number of target variables, this method can be result in large BDDs that may be infeasible to compute.

Negative Output Constraints

We can set up an analagous invariant for the `lowexpr` values in the symbolic simulation. We will then be able to prove that if L is the lowexpr of $s(C, T)$, then $(L[X, C] \wedge R[X, C, T] \rightarrow \neg s(C, T))$.

We can then prove output constraints of the form $\forall C \forall T(P[T, C] \rightarrow \neg s(C, T))$ in a similar manner to the highexpr case.

For our purposes where we use just need to prove that a certain wire is always high, we don't need this component. That being said, analysis of $L[X, C]$ is important for finding weak disagreements and counter examples.

Counter Example Analysis

Suppose that we have $P^R \wedge L \neq \mathrm{False}$. Let (X, C) be such that $(P^R \wedge L)[X, C]$ is true. We show that (X, C) is a counter example to the property $\forall C \forall T(P(C, T) \rightarrow s(C, T))$.

We select some T^* such that $R[X, C, T^*]$ is true. This must exist since $P^R = \mathrm{dom}(R) \wedge \forall T(R[X, C, T] \rightarrow P[C, T])$ so (X, C) is in the domain of R, meaning that $\exists T R[X, C, T]$.

Now since $\forall T(R[X, C, T] \rightarrow P[C, T])$, we must also have that $P[C, T^*]$ is true.

The symbolic simulation invariant for negative properties says that $L[X, C] \wedge R[X, C, T] \rightarrow \neg s(C, T)$. Since $L[X, C]$ and $R[X, C, T^*]$ are true, then $\neg s(C, T^*)$ is true.

By considering the example (C, T^*), we have $\exists C \exists T(P(C, T) \wedge \neg s(C, T)) \equiv \neg \forall C \forall T(P(C, T) \rightarrow s(C, T))$ being true, so the property is disproven.

It is practical to note that that the set of counter examples we find is $\{(C, T) \mid \exists X(R[X, C, T] \wedge L[X, C] \wedge P^R[X, C])\}$ which is described by the image operation on $P^R \wedge L$, $\mathrm{im}(P^R \wedge L, R)$.

We can prove that the counter example analysis for negative properties is correct in a similar manner.

Converting Relational Constraints to Functional Constraints

Traditionally STE only deals with functional properties. However, we can employ a trick to also check relational properties.

Our relational properties are initially written as SytemVerilog assertions. When these assertions are loaded into Jasper Gold, they can be treated as pseudo-signals. Jasper Gold internally treats these properties as additional circuitary that is added onto the specification circuit and has an output signal that corresponds to the correctness of the property at each time.[3]

This means that for us, we can treat these properties as functional properties that just correspond to checking if the pseudo-wire is true. We call this pseudo-wire as the "**property wire**".

[3] We can view the simulated values of these pseudo-wires with `check_symsim -sequence $eval_seq -get $assertions -verbose`.

Properties of this form are actually functional properties and thus can be written in the above 4 tupe form easily as `cout = [(property_wire, k:k, TRUE, FALSE)]`. We can prove this for k and then use the time shift to prove the rest of of the ticks > k. We choose k such that k is the first tick in which we can prove the property to be true.

Indexing Transformations for Relational Properties
An additional benefit from this encoding method is that the property guards are extremely simple and thus have some nice properties related to the indexing transformation.

To check the properties, we would usually apply the weak preimage image operation on the guards. However, we have the following:
- $\mathrm{True}_R = \mathrm{True}^R = \mathrm{dom}(R)[X, C]$
- $\mathrm{False}_R = \mathrm{False}^R = \mathrm{False}$

This means that we only need to take 1 single preimage operation to get the domain and use that for checking all the various properties written in this form. In fact, computing the domain of an indexing relation generated from our automatic abstraction algorithm is a very simple operation that will be described later.

Furthermore, analysis of counter examples is also simplified.

$$\mathrm{True}^R \wedge L = \mathrm{dom}(R)[X, C] \wedge L = L$$

The 2nd equality comes from the fact that we take the strong preimage of the antecedents, so for any signal and time step s, H_s and L_s do not include any cases that are not in the domain of R. I.e. $H_s \vee L_s \rightarrow \mathrm{dom}(R)$. This can be proven with induction over the circuit in the same manner as the symbolic simulation invariants.

So any time L is not false, we have found a counter example. Specifically, any T, C in $\mathrm{im}(L, R)$ is a counter example.

Simplifying Properties for Automatic Abstraction
While we don't need any special form of properties with a manually crafted indexing relation, doing so can be very helpful for doing automatic abstraction. While the traditional way of doing abstraction works by forming the abstractions on all outputs of the specification circuit, we can sometimes derive better results by performing the abstraction algorithm directly from the wire that represents the property.

To assist with this process, we observe that each SV assertion can have its logic be shifted out of the property and into the surrounding specification circuit.

This means that we only need to check properties that are of the form `##k signal_name` where k is the number of clock cycles required to establish a certain property and `signal_name` is the name of the signal that corresponds to the property being true.

Since this signal is a real signal is jasper gold (as compared to the property wire), we can then run the automatic abstraction algorithm directly from this wire to find the indexing relation that will cover all the cases where the property wire is true.

Automatic Abstraction

The original automatic abstraction algorithm from 2007 performed a recursive back propagation from the functional output of the specification to automatically create an indexing relation. By calling `bp(C, specOutput, x_0, not x_0, name)`, we would produce an abstraction that considered all the cases that caused the `specOutput` to be `true` or `false`, while keeping all the signals in `C` as symbolic constants.

The cost savings of the indexing relation came from only using 1 BDD variable on `XNOR` gates and doing a binary encoding on AND gates with ≥ 3 inputs.

- For the XNOR gate, we only needed to consider if it was true or false, and it was true by having both inputs equal and false by having both inputs different.
- For the AND gate with n inputs, we have $n + 1$ cases. Either any of the inputs are false, i.e. the first n cases, or all of them are true, the last case. This would be constrasted with the 2^n cases that would be required if we did not use the indexing relation.

This has been reimplemented and improved.

Application onto Circuits or Specifications

The traditional way of applying the automatic abstraction algorithm is to run it on the output of the specification circuit. This should produce an indexing relation that covers all cases that cause the output of the specification to either be true or false.

Application onto Property Wires

Another way we can use this is to specifically look at all possible cases that are required for a property to be true and ensure that we abstract over them in a manner that we can prove the property being true. This leads us to running the automatic abstraction algorithm on the property wire itself.

Since we are looking specifically at properties of the form "the property wire is true", we don't need to be concerned with considering any indexing case where the property wire is false (because there shouldn't be any). This means that we should run

```
bp(C, property_wire, TRUE, FALSE, name)
```

To generate the required indexing relation.

For this to work, the property wire must be (represented by) a real Jasper Gold wire, which can be done by shifting the logic for the property wire out of the property and into the surrounding circuit as described above.

Partitioned Abstraction Preimage

The automatic abstraction algorithm used here produces a partitioned abstraction of the form

```
[(expr = targVar / Cexpr, hexpr, lexpr)]
```

This is a representation of the indexing relation $\bigwedge(\text{hexpr} \to \text{expr} \wedge \text{lexpr} \to \overline{\text{expr}})$ where `expr` is either some **target variable** or an **expression of symbolic constants** while `hexpr` and `lexpr` are in terms of only the indexing variables.

This representation allows more efficient computation of preimages. We first normalise R into

$$R = S[X, C] \wedge U[X, T]$$

where $U[X, T] = \bigwedge_{t_i \in \text{ TargVars}} (h_i \to t_i \wedge l_i \to \overline{t_i})$

We can then make several observations.

Firstly,

$$\text{dom}(R)[X, C] = S \wedge \bigwedge_i \overline{h_i \wedge l_i}$$

Proof:

$$
\begin{aligned}
\mathrm{dom}(R)[X, C] &= \exists T R[X, C, T] \\
&= S[X, C] \wedge \exists T U[X, T] \\
&= S[X, C] \wedge \bigwedge_i (\exists t_i (h_i \to t_i \wedge l_i \to \overline{t_i})) \\
&= S[X, C] \wedge \bigwedge_i (((h_i \to 1) \wedge (l_i \to 0)) \vee ((h_i \to 0 \wedge l_i \to 1))) \\
&= S[X, C] \wedge \bigwedge_I (\neg l_i \vee \neg h_i) \\
&= S \wedge \bigwedge_i \overline{h_i \wedge l_i}
\end{aligned}
$$

Thus we are able to compute the domain of the indexing relation easily.

Secondly, to compute the preimage of some guard $P[C, T]$, we let $R \downarrow P$ denote that the part of the indexing relation that mentions target variables present in P. Specifically, if $\mathcal{F} = \mathrm{FreeTargVars}(P)$ then

$$
R \downarrow P = S[X, C] \wedge \bigwedge_{t_i \in \mathcal{F}} (h_i \to t_i \wedge l_i \to \overline{t_i})
$$

We have that $P_R = \mathrm{dom}(R) \wedge P_{R \downarrow P}$

Proof:

$$
\begin{aligned}
P_{R[X,C]} &= \exists T (R[X, C, T] \wedge P[C, T]) \\
&= \exists T \left(S[X, C] \wedge \bigwedge_{t_i \notin \mathcal{F}} (h_i \to t_i \wedge l_i \to \overline{t_i}) \wedge \bigwedge_{t_i \in \mathcal{F}} (h_i \to t_i \wedge l_i \to \overline{t_i}) \wedge P[C, T] \right) \\
&= S[X, C] \wedge \exists T_{\neg \mathcal{F}} \left(\bigwedge_{t_i \notin \mathcal{F}} (h_i \to t_i \wedge l_i \to \overline{t_i}) \right) \\
&\quad \wedge \exists T_{\mathcal{F}} \left(\bigwedge_{t_i \in \mathcal{F}} (h_i \to t_i \wedge l_i \to \overline{t_i}) \wedge P[C, T] \right) \\
&= S[X, C] \wedge \exists T_{\neg \mathcal{F}} \left(\bigwedge_{t_i \notin \mathcal{F}} (h_i \to t_i \wedge l_i \to \overline{t_i}) \right) \wedge \exists T_{\mathcal{F}} \left(\bigwedge_{t_i \in \mathcal{F}} (h_i \to t_i \wedge l_i \to \overline{t_i}) \right) \\
&\quad \wedge S[X, C] \wedge \exists T_{\mathcal{F}} \left(\bigwedge_{t_i \in \mathcal{F}} (h_i \to t_i \wedge l_i \to \overline{t_i}) \wedge P[C, T] \right) \\
&= S[X, C] \wedge \exists T \left(\bigwedge_{t_i \in \text{ targVars}} (h_i \to t_i \wedge l_i \to \overline{t_i}) \right) \\
&\quad \wedge \exists T_{\mathcal{F}} \left(S[X, C] \wedge \bigwedge_{t_i \in \mathcal{F}} (h_i \to t_i \wedge l_i \to \overline{t_i}) \wedge P[C, T] \right) \\
&= S[X, C] \wedge \bigwedge_i \left(\overline{h_i \wedge l_i} \right) \wedge P_{R \downarrow P} \\
&= \mathrm{dom}(R) \wedge P_{R \downarrow P}
\end{aligned}
$$

Now, we can even further consider common cases of P that we will be constructing preimages for.

$$P_R = \begin{cases} \text{dom}(R) \wedge P \text{ if } \mathcal{F} \cap \text{TargVars} = \emptyset \\ \text{dom}(R) \wedge \overline{l_i} \text{ if } P = t_i \\ \text{dom}(R) \wedge \overline{h_i} \text{ if } P = \overline{t_i} \\ \text{dom}(R) \wedge P_{R \downarrow P} \text{ otherwise} \end{cases}$$

Proof:

If $\mathcal{F} \cap \text{TargVars} = \emptyset$, $P_{R\downarrow P} = \text{True}$ so

$$P_R = \text{dom}(R) \wedge \exists T(P[C,T]) = \text{dom}(R) \wedge P$$

If $P = t_i$, $P_{R\downarrow P} = (h_i \rightarrow t_i) \wedge (l_i \rightarrow \overline{t_i})$, so

$$P_R = \text{dom}(R) \wedge \exists t_i((h_i \rightarrow t_i) \wedge (l_i \rightarrow \overline{t_i} \wedge t_i) \wedge t_i) = \text{dom}(R) \wedge \overline{l_i}$$

The $P = \overline{t_i}$ case is analagous to the above.

Checking Properties under Environmental Constraints

Environmental constraints are constraints on the inputs to the circuit. They are of the form $P[C,T]$ to denote that we only need a constraint to hold if $P[C,T]$ is true. We call such a constraint a "care predicate".

Index Over Care Predicate Cases

Notice that one easy way to include environmental constraints is to simply add them to the guards of the output constraint. We can then check for the property holding under the enviromental constraint as described above.

However, this can lead to problems for abstractions that merge cases covering $(C_1, T_1), (C_2, T_2)$ where $C_1, T_1 \vDash P$ but $C_2, T_2 \nvDash P$. Specifically, are more likely to have weak disagreements as these cases cannot assume the environmental constraint holds. This is referenced to in the 2002 paper where it is recommended to find indexing relations that exactly index cases in P and not any in $\neg P$. It is recommended to find an indexing relation specific to each particular constraint that avoids covering unnecessary cases not in P.

Another way to mitigate the weak disagreements is to use the alternative checking method involving reversing the indexing, described earlier. This can avoid weak disagreements in some cases but can be more computationally expensive. This is especially true if we use the automatic abstraction algorithm since we cannot make use of the partitioned abstraction preimage.

Parametric Encoding

An alternative approach would be to use the `param` function to encode the care predicate into the indexing relation, the antecedent and the output constraints. Details in the 2002 paper.

This has the problem where the partitioned indexing relation structure is destroyed during the folding of the param substitutions into the indexing relation.

An alternative similar strategy would involve doing the parameterisation first and then doing the automatic abstraction. However, that would make it difficult to use the symbolic constants effectively since we would need to specify that in terms of the new input signals from the param substitution. If the input constraints don't involve the signals that were meant to be the symbolic constants, it is possible to only apply param on the other input signals and leave the symbolic constants as is.